Selection Sort

Data Structures Using C – Sorting Algorithms

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

Recall: Sorting Algorithms

We have already discussed:

- Bubble Sort
- Insertion Sort

Bubble Sort

Repeatedly compare and swap adjacent elements.

Insertion Sort

Insert each element into its correct position in the sorted portion.

Today's Topic

Selection Sort repeatedly selects the smallest element from the unsorted portion.

What is Selection Sort?

Selection Sort is a simple comparison-based sorting algorithm.

- It divides the array into sorted and unsorted portions.
- It finds the smallest element in the unsorted portion.
- It places that element at the beginning of the unsorted portion.
- The sorted portion grows after each pass.

Main Idea

Select Minimum $\rightarrow$ Swap $\rightarrow$ Repeat

Real-Life Analogy

Suppose five students have obtained the following marks:

$$[65,\ 42,\ 89,\ 31,\ 74]$$

We want to arrange them in ascending order.

1. Find the smallest mark: 31.
2. Place it in the first position.
3. Find the smallest mark among the remaining values.
4. Place it in the second position.
5. Continue until the list is sorted.

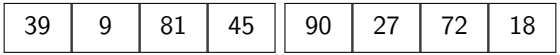
$$\boxed{31,\ 42,\ 65,\ 74,\ 89}$$

Selection Sort: Two Partitions

Consider:

$$A = [39,\ 9,\ 81,\ 45,\ 90,\ 27,\ 72,\ 18]$$

Initially:



Sorted Portion

Initially empty.

Unsorted Portion

Initially contains all eight elements.

Selection Sort: Working Principle

For an array containing n elements:

1. Set the current position to index 0.
2. Find the minimum element from index 0 to $n-1$.
3. Swap it with the element at index 0.
4. Move the current position to index 1.
5. Find the minimum element in the remaining portion.
6. Repeat until the array is sorted.

Important

After Pass i, the first i elements are in their correct positions, using one-based pass numbering.

Simple Example: Initial Array

Consider:

$$A = [29,\ 10,\ 14,\ 37,\ 13]$$

29	10	14	37	13

- Current position: 0
- Minimum value: 10
- Minimum index: 1

Swap:

$$A[0] \leftrightarrow A[1]$$

Selection Sort: Pass 1

Initial:

$$[29,\ 10,\ 14,\ 37,\ 13]$$

Minimum:

$$MIN = 10$$

Swap 29 and 10.

After Pass 1:

10	29	14	37	13

$$\underbrace{[10]}_{\text{Sorted}} \quad \underbrace{[29,\ 14,\ 37,\ 13]}_{\text{Unsorted}}$$

Selection Sort: Pass 2

Current array:

$$[10,\ 29,\ 14,\ 37,\ 13]$$

Search in:

$$[29,\ 14,\ 37,\ 13]$$

Minimum:

$$MIN = 13$$

Swap 29 and 13.

After Pass 2:

$$\boxed{[10,\ 13,\ 14,\ 37,\ 29]}$$

Selection Sort: Pass 3

Current array:

$$[10,\ 13,\ 14,\ 37,\ 29]$$

Unsorted portion:

$$[14,\ 37,\ 29]$$

Minimum:

$$MIN = 14$$

The minimum is already in its correct position.

After Pass 3:

$$\boxed{[10,\ 13,\ 14,\ 37,\ 29]}$$

No swap is required.

Selection Sort: Pass 4

Current array:

$$[10,\ 13,\ 14,\ 37,\ 29]$$

Remaining elements:

$$[37,\ 29]$$

Minimum:

$$MIN = 29$$

Swap 37 and 29.

Final sorted array:

$$\boxed{[10,\ 13,\ 14,\ 29,\ 37]}$$

Simple Example: Complete Trace

Pass	Minimum	Array After Pass
Initial	–	29, 10, 14, 37, 13
1	10	10, 29, 14, 37, 13
2	13	10, 13, 14, 37, 29
3	14	10, 13, 14, 37, 29
4	29	10, 13, 14, 29, 37

Number of passes $= n - 1 = 4$

Textbook Example: Selection Sort

Consider the array:

$$A = [39,\ 9,\ 81,\ 45,\ 90,\ 27,\ 72,\ 18]$$



- Total elements: $n = 8$
- Total passes: $n - 1 = 7$
- Sorting order: Ascending

Textbook Example: Pass 1

Initial:

$$[39,\ 9,\ 81,\ 45,\ 90,\ 27,\ 72,\ 18]$$

Minimum value:

$$MIN = 9, \quad POS = 1$$

Swap:

$$A[0] \leftrightarrow A[1]$$

After Pass 1:

$$\boxed{[9,\ 39,\ 81,\ 45,\ 90,\ 27,\ 72,\ 18]}$$

Textbook Example: Pass 2

Current array:

$$[9,\ 39,\ 81,\ 45,\ 90,\ 27,\ 72,\ 18]$$

Search minimum in:

$$[39,\ 81,\ 45,\ 90,\ 27,\ 72,\ 18]$$

Minimum:

$$MIN = 18, \quad POS = 7$$

Swap:

$$A[1] \leftrightarrow A[7]$$

After Pass 2:

$$\boxed{[9,\ 18,\ 81,\ 45,\ 90,\ 27,\ 72,\ 39]}$$

Textbook Example: Pass 3

Current array:

$$[9,\ 18,\ 81,\ 45,\ 90,\ 27,\ 72,\ 39]$$

Unsorted portion:

$$[81,\ 45,\ 90,\ 27,\ 72,\ 39]$$

Minimum:

$$MIN = 27, \quad POS = 5$$

Swap 81 and 27.
After Pass 3:

$$\boxed{[9,\ 18,\ 27,\ 45,\ 90,\ 81,\ 72,\ 39]}$$

Textbook Example: Pass 4

Current array:

$$[9,\ 18,\ 27,\ 45,\ 90,\ 81,\ 72,\ 39]$$

Remaining unsorted portion:

$$[45,\ 90,\ 81,\ 72,\ 39]$$

Minimum:

$$MIN = 39, \quad POS = 7$$

Swap 45 and 39.
After Pass 4:

$$\boxed{[9,\ 18,\ 27,\ 39,\ 90,\ 81,\ 72,\ 45]}$$

Textbook Example: Pass 5

Current array:

$$[9,\ 18,\ 27,\ 39,\ 90,\ 81,\ 72,\ 45]$$

Remaining unsorted portion:

$$[90,\ 81,\ 72,\ 45]$$

Minimum:

$$MIN = 45, \quad POS = 7$$

Swap 90 and 45.
After Pass 5:

$$\boxed{[9,\ 18,\ 27,\ 39,\ 45,\ 81,\ 72,\ 90]}$$

Textbook Example: Pass 6

Current array:

$$[9,\ 18,\ 27,\ 39,\ 45,\ 81,\ 72,\ 90]$$

Remaining unsorted portion:

$$[81,\ 72,\ 90]$$

Minimum:

$$MIN = 72, \quad POS = 6$$

Swap 81 and 72.
After Pass 6:

$$\boxed{[9,\ 18,\ 27,\ 39,\ 45,\ 72,\ 81,\ 90]}$$

Textbook Example: Pass 7

Current array:

$$[9,\ 18,\ 27,\ 39,\ 45,\ 72,\ 81,\ 90]$$

Remaining unsorted portion:

$$[81,\ 90]$$

Minimum:

$$MIN = 81$$

Already in correct position.

Final sorted array:

$$\boxed{[9,\ 18,\ 27,\ 39,\ 45,\ 72,\ 81,\ 90]}$$

Textbook Example: Complete Trace

Pass	Minimum	Array After Pass
Initial	–	39, 9, 81, 45, 90, 27, 72, 18
1	9	9, 39, 81, 45, 90, 27, 72, 18
2	18	9, 18, 81, 45, 90, 27, 72, 39
3	27	9, 18, 27, 45, 90, 81, 72, 39
4	39	9, 18, 27, 39, 90, 81, 72, 45
5	45	9, 18, 27, 39, 45, 81, 72, 90
6	72	9, 18, 27, 39, 45, 72, 81, 90
7	81	9, 18, 27, 39, 45, 72, 81, 90

9, 18, 27, 39, 45, 72, 81, 90

Selection Sort: Algorithm

SELECTION_SORT(A, N)

1. Repeat for $i = 0$ to $N - 2$.
2. Set $POS = i$.
3. Repeat for $j = i + 1$ to $N - 1$.
4. If $A[j] < A[POS]$, set $POS = j$.
5. End inner loop.
6. If $POS \neq i$, swap $A[i]$ and $A[POS]$.
7. End outer loop.

Key Observation

The algorithm selects the smallest element from the unsorted portion during each pass.

Selection Sort: Loop Variables

Variable	Purpose
i	Current position to be filled
j	Scans the unsorted portion
POS	Index of the smallest element found
$TEMP$	Temporary variable for swapping
N	Total number of elements

For every pass:

$$POS = i$$

Then:

$$j = i + 1, \ldots, N - 1$$

Selection Sort: C Function

```c
void selectionSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
        {
            if (a[j] < a[minIndex])
                minIndex = j;
        }

        if (minIndex != i)
        {
            int temp = a[i];
            a[i] = a[minIndex];
            a[minIndex] = temp;
        }
    }
}
```

Selection Sort: Complete C Program

```c
#include <stdio.h>

void selectionSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minIndex])
                minIndex = j;

        if (minIndex != i)
        {
            int temp = a[i];
            a[i] = a[minIndex];
            a[minIndex] = temp;
        }
    }
}

int main(void)
{
    int a[] = {39, 9, 81, 45, 90, 27, 72, 18};
    int n = sizeof(a) / sizeof(a[0]);
```

Selection Sort: Expected Output

Input:

```
39 9 81 45 90 27 72 18
```

Output:

```
9 18 27 39 45 72 81 90
```

Explanation

The smallest remaining element is selected and placed at its correct position during each pass.

Selection Sort: Number of Comparisons

For n elements:

Pass	Comparisons
1	$n-1$
2	$n-2$
3	$n-3$
$\vdots$	$\vdots$
$n-1$	1

Total comparisons:

$$C(n) = (n-1) + (n-2) + \cdots + 1$$

$$\boxed{C(n) = \frac{n(n-1)}{2}}$$

Selection Sort: Time Complexity

$$C(n) = \frac{n(n-1)}{2}$$

Expanding:

$$C(n) = \frac{n^2 - n}{2}$$

For large n, the quadratic term dominates.
Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

Important

Selection Sort performs the same number of key comparisons regardless of the initial order of the array.

Best, Average and Worst Cases

Case	Time Complexity
Best	$\Theta(n^2)$
Average	$\Theta(n^2)$
Worst	$\Theta(n^2)$
Auxiliary space	$O(1)$

- Best case: already sorted array.
- Worst case: reverse-sorted array.
- The minimum still needs to be searched in every remaining subarray.

Selection Sort: Number of Swaps

Selection Sort performs at most one swap per pass.
For n elements:

$$\boxed{\text{Maximum swaps} = n - 1}$$

For example, if $n = 8$:

$$\text{Maximum swaps} = 7$$

Advantage

Selection Sort performs relatively few swaps compared with Bubble Sort.
This can be useful when writing or moving records is expensive.

Is Selection Sort Stable?

Consider:

$$[(4, A),\ (4, B),\ (2, C)]$$

The number represents the sorting key.

Initially:

$$(4, A),\ (4, B),\ (2, C)$$

Selection Sort finds the minimum key 2 and swaps it with the first element.
Result:

$$(2, C),\ (4, B),\ (4, A)$$

Observation

The relative order of the two records with key 4 has changed.
Therefore, standard Selection Sort is **not stable**.

Properties of Selection Sort

Property	Selection Sort
Comparison-based	Yes
In-place	Yes
Stable	No (standard version)
Adaptive	No
Best-case time	$\Theta(n^2)$
Worst-case time	$\Theta(n^2)$
Auxiliary space	$O(1)$
Maximum swaps	$n-1$

Advantages and Limitations

Advantages

- Easy to understand.
- Simple implementation.
- Constant auxiliary space.
- At most $n - 1$ swaps.
- Useful for small datasets.

Limitations

- Quadratic time complexity.
- Not adaptive.
- Usually not stable.
- Inefficient for large datasets.
- Does not benefit significantly from nearly sorted input.

Bubble vs Insertion vs Selection Sort

Property	Bubble	Insertion	Selection
Best time	$O(n)$*	$O(n)$	$O(n^2)$
Average time	$O(n^2)$	$O(n^2)$	$O(n^2)$
Worst time	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space	$O(1)$	$O(1)$	$O(1)$
Stable	Yes	Yes	No
In-place	Yes	Yes	Yes
Adaptive	Yes*	Yes	No
Main operation	Swap	Shift	Select minimum

* Bubble Sort is adaptive and has linear best-case time when early termination is implemented.

Classroom Activity 1

Sort the following array using Selection Sort:

$$[64,\ 25,\ 12,\ 22,\ 11]$$

For every pass:

1. Identify the minimum element.
2. Identify its index.
3. Show the swap operation.
4. Write the updated array.

Additional Question:
How many comparisons will be performed?

Classroom Activity 1: Solution

Pass	Minimum	Array After Pass
Initial	–	64, 25, 12, 22, 11
1	11	11, 25, 12, 22, 64
2	12	11, 12, 25, 22, 64
3	22	11, 12, 22, 25, 64
4	25	11, 12, 22, 25, 64

Total comparisons:

$$\frac{5(5-1)}{2} = 10$$

Classroom Activity 2

Consider:

$$A = [10,\ 20,\ 30,\ 40,\ 50]$$

Answer the following:

1. Is the array already sorted?
2. How many comparisons will Selection Sort perform?
3. How many actual swaps are needed if self-swaps are skipped?
4. What is the time complexity?
5. Would Insertion Sort perform better on this input? Why?

Classroom Activity 2

Consider:

$$A = [10,\ 20,\ 30,\ 40,\ 50]$$

Answer the following:

1. Is the array already sorted?
2. How many comparisons will Selection Sort perform?
3. How many actual swaps are needed if self-swaps are skipped?
4. What is the time complexity?
5. Would Insertion Sort perform better on this input? Why?

Expected Answers

Yes; 10 comparisons; 0 swaps; $\Theta(n^2)$; Insertion Sort requires only $\Theta(n)$ time.

MCQ 1

What does Selection Sort select during each pass for ascending order?

1. Largest element
2. Smallest element
3. Middle element
4. Random element

MCQ 1

What does Selection Sort select during each pass for ascending order?

1. Largest element
2. Smallest element
3. Middle element
4. Random element

Answer

Option 2: Smallest element.

MCQ 2

What is the worst-case time complexity of Selection Sort?

1. $O(1)$
2. $O(n)$
3. $O(n \log n)$
4. $O(n^2)$

MCQ 2

What is the worst-case time complexity of Selection Sort?

1. $O(1)$
2. $O(n)$
3. $O(n \log n)$
4. $O(n^2)$

Answer

Option 4: $O(n^2)$.

MCQ 3

Which statement about standard Selection Sort is correct?

1. It is stable.
2. It requires $O(n)$ auxiliary space.
3. It performs at most $n - 1$ swaps.
4. It always runs in $O(n)$ time.

MCQ 3

Which statement about standard Selection Sort is correct?

1. It is stable.
2. It requires $O(n)$ auxiliary space.
3. It performs at most $n - 1$ swaps.
4. It always runs in $O(n)$ time.

Answer

Option 3: At most $n - 1$ swaps.

MCQ 4

Which sorting algorithm is generally preferable for a nearly sorted small array?

1. Selection Sort
2. Insertion Sort
3. Both always perform identically
4. Neither algorithm can sort the array

MCQ 4

Which sorting algorithm is generally preferable for a nearly sorted small array?

1. Selection Sort
2. Insertion Sort
3. Both always perform identically
4. Neither algorithm can sort the array

Answer

Option 2: Insertion Sort.

Practice Questions

1. Explain Selection Sort with an example.
2. Write its algorithm and C program.
3. Trace the sorting of:

$$[29,\ 10,\ 14,\ 37,\ 13]$$

4. Derive the number of comparisons.
5. Explain why Selection Sort is not stable.
6. Compare Selection Sort with Bubble Sort.
7. Compare Selection Sort with Insertion Sort.
8. Explain its advantages and limitations.

Lecture Summary

Key Concepts

- Selection Sort repeatedly selects the minimum.
- The sorted portion grows from left to right.
- Each pass fixes one element.
- Total comparisons:

$$\frac{n(n-1)}{2}$$

- Time complexity: $\Theta(n^2)$.
- Auxiliary space: $O(1)$.
- Standard Selection Sort is not stable.

References

1. Reema Thareja, *Data Structures Using C*, chapter on Searching and Sorting.
2. Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
3. Thomas H. Cormen et al., *Introduction to Algorithms*.

Lecture Feedback

Your Feedback Matters

Please share your feedback on:

- Selection Sort concepts
- Minimum element selection
- Step-by-step dry runs
- C implementation
- Complexity analysis



Scan to Submit Feedback

Thank You

Questions and Discussion

Select Minimum $\rightarrow$ Swap $\rightarrow$ Repeat